Modul 16 — MEPRO (Manufacturing Execution System)

Memantau mesin pabrik secara real-time lewat sensor ESP32: output, status, downtime, reject, Work Order, dan OEE

---

1. Informasi Project

Item	Keterangan
Nama	MEPRO — Manufacturing Execution System (paket Fase 1–6)
Bentuk paket	**Overlay**: berkas yang disalin ke atas proyek Laravel baru (folder `mepro-overlay`)
Jenis	SPA (Laravel 11/12 + Vue 3 + Vue Router) dengan REST API, WebSocket, dan **perangkat IoT** (firmware ESP32)
Peran pengguna	`admin`, `supervisor`, `operator`
Dokumen	`docs/01-architecture.md`, `02-mqtt-contract.md`, `03-erd.md`, `04-api.md`, `05-oee.md`, `06-hardening.md`, `TEST-CHECKLIST.md`
Akun demo	`admin@mepro.test`, `supervisor@mepro.test`, `operator@mepro.test` (password `password`)
Status pengujian (menurut dokumennya)	Logika murni (OEE, shift, parsing MQTT) lulus **27 unit test**; frontend berhasil di-build; seluruh PHP lolos `php -l`; kontrak MQTT diuji simulator → broker → parser. **Belum diuji:** menjalankan Laravel penuh dan firmware di perangkat nyata

2. Tujuan Pembuatan

1. Mengganti pencatatan output mesin manual (kertas/Excel) dengan **data otomatis dari sensor**.

2. Menampilkan kondisi lantai produksi **detik demi detik** (berjalan, diam, downtime, offline).
3. Menghitung **OEE** (Availability × Performance × Quality) yang objektif per run, mesin, dan operator.
4. Mengelola Work Order dari perencanaan sampai selesai, termasuk BOM dan reject QC.
5. Menyediakan antarmuka sederhana untuk operator (HMI layar sentuh) dan integrasi dengan ERP.

3. Manfaat

- **Tahu mesin mana yang berhenti sekarang juga**, bukan besok pagi saat laporan tiba.
- **Downtime otomatis terdeteksi** lalu operator memberi alasannya → analisis Pareto penyebab kerugian.
- **OEE yang adil:** downtime terencana (istirahat, setup) tidak menghukum operator.
- **Output akurat:** data dari sensor, bukan tebakan atau catatan akhir shift.
- **Tahan gangguan jaringan:** firmware menyimpan data di buffer saat Wi-Fi putus dan mengirim ulang; pesan ganda tidak terhitung dua kali.
- **Siap dihubungkan ke ERP** lewat endpoint upsert Work Order berkunci API.

> **Mengapa penting?** Pabrik tidak bisa memperbaiki apa yang tidak terukur. MEPRO mengubah suara mesin menjadi angka yang bisa dipertanggungjawabkan.

4. Kelebihan & Kekurangan

Kelebihan

- **Kontrak MQTT jelas dan berversi** (`mepro/v1/machines/{kode}/{counter|status|heartbeat}`) dengan `seq`, `ts`, dan `delta`.
- **Idempoten:** unik `(device_id, seq)` + `insertOrIgnore` → kirim ulang aman; pesan terlambat tetap dihitung ke output sesuai `ts`.
- **Satu run aktif per mesin** dijaga *partial unique index* PostgreSQL.
- Semua waktu disimpan **UTC** (`timestamptz`) dan ditampilkan Asia/Jakarta → bebas bug lintas shift.

- Status mesin dihitung terpusat (`MachineStatusService`) agar konsisten: offline / downtime / running / idle.
- Output run = jumlah delta pada rentang waktunya dan **di-cache saat run ditutup**, tetap benar walau data mentah sudah diringkas.
- Retensi otomatis: data mentah > 30 hari diringkas per jam.
- Tersedia **simulator** (`tools/simulator.py`) dengan uji buffer dan uji duplikat, sehingga bisa dicoba tanpa hardware.
- Pedoman hardening produksi (supervisor, cron, ACL Mosquitto, TLS, TimescaleDB opsional).

Kekurangan

- Channel Reverb `mepro` **publik** (tanpa login) demi kesederhanaan; hanya berisi hitungan dan status, dan `docs/06-hardening.md` menjelaskan cara menjadikannya privat.
- **Buffer offline firmware ada di RAM**: Wi-Fi putus aman, tetapi listrik padam menghilangkan buffer (rekomendasi: simpan ke LittleFS).
- Firmware memakai PubSubClient sehingga publish hanya **QoS 0** (celah dapat dideteksi lewat `seq`).
- UI memakai CSS biasa, bukan Tailwind.
- Instalasi berupa overlay: butuh beberapa langkah manual (membuat proyek Laravel, menyalin berkas, mengatur `.env`).
- Membutuhkan Docker (atau PostgreSQL + broker MQTT terpisah), serta perangkat keras ESP32 + sensor untuk pemakaian nyata.
- Mode anonim Mosquitto di pengembangan harus dimatikan di produksi.

5. Stack Teknologi

Lapisan	Teknologi
Bahasa	PHP 8.2+, JavaScript, SQL, C++ (Arduino), Python (simulator)
Backend	Laravel 11/12, Laravel Sanctum (token Bearer), Laravel Reverb (WebSocket), `php-mqtt/client`
Frontend	Vue 3, Vue Router, Axios, Laravel Echo + `pusher-js` , `@vitejs/plugin-vue` , CSS biasa

Lapisan	Teknologi
Database	PostgreSQL 16 (image TimescaleDB), `timestamptz`, partial unique index
Pesan	MQTT broker **Eclipse Mosquitto 2** (alternatif EMQX)
IoT	ESP32 (Arduino framework via **PlatformIO**), `knolleary/PubSubClient` ^2.8, NTP, NVS, sensor proximity/optocoupler
Infrastruktur	Docker Compose, Supervisor, cron
Pengujian	PHPUnit (OEE, shift resolver, MQTT parsing), simulator Python (`paho-mqtt`)

Yang harus disiapkan

PHP 8.2+ (`pgsql`, `pdo_pgsql`, `mbstring`, `pcntl` disarankan), Composer, Node.js 20+, **Docker** (Postgres + Mosquitto), Python 3 + `pip install paho-mqtt`, (untuk perangkat) **PlatformIO** + papan ESP32 + sensor + catu daya, dan editor kode.

6. Cara Menjalankan (Detail)

```
# 1. Proyek Laravel baru
composer create-project laravel/laravel mepro && cd mepro

# 2. Infrastruktur: salin docker-compose.yml dan folder docker/ dari paket, lalu
docker compose up -d            # Postgres (5432) + Mosquitto (1883)

# 3. API token, broadcasting, MQTT client
php artisan install:api
php artisan install:broadcasting   # pilih Reverb, jawab ya untuk dependensi node
composer require php-mqtt/client

# 4. Salin seluruh isi overlay ke root proyek (timpa User.php, routes/*, vite.config.js, DatabaseSeeder)
unzip -o mepro-overlay.zip -d /tmp/ov && cp -r /tmp/ov/mepro-overlay/. .
```

```
# 5. Frontend
npm install vue vue-router @vitejs/plugin-vue axios laravel-echo pusher-js

# 6. .env dari .env.mepro.example (DB pgsql, MQTT, APP_TIMEZONE=UTC, MEPRO_*)
#    config/database.php: pada koneksi 'pgsql' tambahkan  'timezone' => 'UTC'

# 7. Database
php artisan migrate --seed
```

Menjalankan (5 terminal):

```
php artisan serve              # http://127.0.0.1:8000
php artisan reverb:start       # WebSocket real-time
php artisan mqtt:listen        # penerima data mesin
php artisan schedule:work      # cek offline + downtime tiap menit
npm run dev                    # Vite
```

Tanpa hardware: `pip install paho-mqtt` lalu `python tools/simulator.py --machines INJ-01,INJ-02` (opsi: `--rate`, `--interval`, `--downtime-after`, `--downtime-for`, `--buffer-test`, `--duplicate-test`).

Cek kesehatan: buka `http://127.0.0.1:8000/api/health` — harus `status: ok` tanpa `warning`.

Uji unit: `php artisan test --testsuite=Unit`.

Parameter `.env`: `APP_TIMEZONE=UTC` (**jangan diubah**), `MEPRO_TIMEZONE=Asia/Jakarta`, `MQTT_HOST/PORT/USERNAME/PASSWORD`, `MEPRO_IDLE_THRESHOLD=120` (detik tanpa hitungan = downtime), `MEPRO_OFFLINE_AFTER=90`, `MEPRO_RAW_RETENTION_DAYS=30`, `MEPRO_INTEGRATION_KEY` (ganti dengan kunci acak panjang).

Firmware ESP32: salin `firmware/esp32/include/config.example.h` menjadi `config.h`, isi `WIFI_SSID`, `WIFI_PASS`, `MQTT_HOST`, `MQTT_USER`, `MACHINE_CODE` (harus sama dengan `machines.code`), dan `SENSOR_PIN` (bawaan GPIO 27, `INPUT_PULLUP`, aktif LOW); `DEBOUNCE_US` 20 ms, kirim tiap 5 detik, heartbeat tiap 30 detik, buffer 256 entri (≈ 21 menit); unggah dengan PlatformIO.

Produksi: pakai `deploy/supervisor/mepro.conf` (mqtt:listen + reverb) dan `deploy/crontab.txt`; jalankan `php artisan config:cache route:cache` dan `npm run build`; matikan `allow_anonymous` di Mosquitto, buat akun per device dengan ACL, aktifkan TLS (port 8883, `MQTT_TLS=true`); backup `pg_dump` harian; pantau `/api/health`.

7. Hubungan Antar Bagian

```
ESP32 ──MQTT──▶ Mosquitto ──▶ php artisan mqtt:listen ──▶ machine_counts + status mesin
                                                                  └─▶ event MachineUpdated ──▶ Reverb ──▶ Vue (Echo, channel publik "mepro")
Operator/Supervisor ──(Sanctum Bearer)──▶ REST /api ──▶ WorkOrderService · DowntimeService · MachineStatusService · RunMetrics
ERP ──(header X-Api-Key)──▶ POST /api/integration/work-orders (upsert Work Order)
Scheduler ─▶ mepro:check-machines (tiap menit: offline + downtime otomatis) · mepro:prune-counts (02:00)
```

Kontrak MQTT v1:

Topic	Arah	Isi
`.../counter`	device → server	Batch hitungan tiap 5 dtk (`seq` , `ts` , `delta` , `total` , `buffered` , `fw`); bila delta = 0 dikirim tiap 60 dtk
`.../status`	device → server (retain)	`online` ; `offline` lewat Last Will
`.../heartbeat`	device → server	Tiap 30 dtk (`ts` , `rssi` , `uptime` , `buffered_count`)

Pesan valid apa pun menandai mesin online; tanpa pesan > 90 detik → offline. Pesan untuk mesin tak terdaftar/nonaktif diabaikan dan dicatat.

Alur Work Order: `planned → running ↔ paused → completed` (atau `cancelled`); satu WO dapat memiliki banyak *run* (per shift/operator/mesin).

Perhitungan OEE (per run):

- loading = durasi − downtime terencana
- operating = loading − downtime tak terencana (termasuk yang belum diberi alasan)
- Availability = operating ÷ loading
- Performance = min(1, ideal cycle × output ÷ operating)
- Quality = (output − reject) ÷ output
- **OEE = A × P × Q** (penggabungan memakai jumlah berbobot waktu/jumlah, bukan rata-rata persen)

8. Fungsi Halaman & Cara Mengoperasikan

8.1 Halaman Web (Vue)

Halaman	Fungsi	Cara pakai
Login	Masuk dengan token Sanctum	Email + password
Dashboard	Pantau semua mesin real-time (status, run aktif, output, OEE, downtime)	Buka di layar TV/monitor lantai produksi; data bergerak otomatis lewat WebSocket
Layar Operator (HMI)	Antarmuka sentuh untuk operator	*Mulai Work Order* → pilih WO dan mesin → **Mulai Produksi**; **Catat Reject (QC)** (pilih alasan + jumlah); **Downtime** (beri alasan pada downtime otomatis, **Mulai Downtime** manual, atau **Mesin jalan lagi**); **Jeda / Akhiri Shift**; **Selesaikan Work Order**
Work Orders	Perencanaan dan status WO	Supervisor membuat, mengubah, membatalkan, atau menyelesaikan WO
Laporan	OEE per run, mesin, operator, serta Pareto downtime/reject	Pilih rentang tanggal (zona Asia/Jakarta) dan mesin
Master Data	Mesin, device, produk, material, shift, alasan reject, alasan downtime, pengguna	Admin menambah/mengubah/menghapus; produk punya BOM (qty per unit + scrap %)

8.2 Hak Akses

Peran	Boleh
`admin`	Semua, termasuk CRUD master data
`supervisor`	Work Order, BOM, laporan OEE, melihat daftar pengguna

Peran	Boleh
`operator`	Memulai run atas nama sendiri, catat reject dan downtime

8.3 Endpoint API Utama (`/api` , header `Authorization: Bearer <token>`)

`POST /login` · `GET /me` · `GET /health` · `GET /machines` · `GET|POST|PUT|DELETE /master/{resource}` · `GET|PUT /products/{id}/bom` · `GET|POST /work-orders` (+ `/start` , `/cancel` , `/complete`) · `POST /runs/{id}/end` · `POST /runs/{id}/rejects` · `POST /machines/{id}/downtimes` · `PATCH /downtimes/{id}` · `GET /reports/oee` · `POST /integration/work-orders` (header `X-Api-Key`).

8.4 Alur Kerja Harian yang Disarankan

1. Supervisor membuat Work Order (produk, target, mesin).
2. Operator membuka **Layar Operator**, memilih WO, menekan **Mulai Produksi**.
3. Mesin berjalan: sensor mengirim hitungan; output dan OEE naik di dashboard.
4. Mesin berhenti > 120 detik: downtime otomatis terbuka; operator memilih alasan.
5. Ada barang cacat: operator mencatat **reject** dengan alasan.
6. Akhir shift: **Jeda / Akhiri Shift**; WO selesai: **Selesaikan Work Order**.
7. Supervisor membuka **Laporan** untuk OEE dan Pareto penyebab kerugian.

8.5 Perangkat Keras (Ringkas)

Sensor proximity/optocoupler → GPIO ESP32 (pakai isolasi antara sensor industri 24V dan GPIO 3,3V), catu daya stabil, enclosure minimal IP65, antena Wi-Fi eksternal bila banyak logam.

9. Kalimat Penutup untuk Calon Pengguna

Setiap detik mesin berhenti adalah produk yang tidak lahir. **MEPRO membuat detik-detik itu terlihat, terhitung, dan bisa diperbaiki.** Mulailah dari satu mesin dan satu sensor — dalam sehari, Anda akan melihat angka OEE pertama yang benar-benar berasal dari lantai produksi Anda sendiri.